

Guion (20 min) — El efecto Mpemba cuántico en HPC

Charla conjunta · versión extendida ≈ 20 min

Juan intro · T1 · redes tensoriales · conclusiones

Sebastián evolución temporal · trayectorias · GPU · relevancia macro

Lenguaje natural. Más detalle que la versión de 10 min: incluye la intuición física, el razonamiento HPC y la lectura de cada figura. Los tiempos suman $\approx 20:00$.

Juan 0 · Apertura (*Título*)

0:20

- “Buenas. Vamos a contar un proyecto de **computación de alto rendimiento** aplicado a un fenómeno físico curioso: el **efecto Mpemba cuántico**. La idea: cuatro tareas numéricas, cada una con su versión *serial* y *paralela*, validadas contra soluciones exactas y con su estudio de escalado.”
- “Nos repartimos: en **teal** lo de Juan, en **naranja** lo de Sebastián.”

Juan 1 · De lo clásico a lo cuántico — La anomalía

1:10

- “El efecto Mpemba clásico es contraintuitivo: bajo ciertas condiciones, el agua **más caliente se congela antes** que la templada. Se observa también en coloides, granulares, imanes...”
- “La clave *no* es la temperatura en sí, sino una idea geométrica: si medimos una **distancia al equilibrio**, la curva que parte **más lejos** puede **cruzar** a la que parte más cerca y llegar antes. Ese *cruce* es la firma del efecto.”
- “Nuestra pregunta: ¿ocurre lo mismo en un sistema **cuántico abierto**? Y la respuesta es sí — y además se puede *predecir* con álgebra lineal.” (señalar la figura del cruce)

Juan 1 · Matemática: GKSL y autovalores

1:40

- “Un sistema cuántico abierto markoviano evoluciona con la **ecuación GKSL**, o de Lindblad: un término coherente (conmutador con H) más un **disipador** que modela el acoplamiento al baño.”
- “Como es *lineal* en ρ , la reescribimos con un operador, el **Liouvilliano** $\mathcal{L}$. Y aquí entra lo importante: diagonalizamos $\mathcal{L}$ y obtenemos sus **autovalores** λ_k , todos con parte real ≤ 0 .”
- “Entonces la evolución es una **suma de modos que decaen**: cada modo con su tasa $|\text{Re } \lambda_k|$. A tiempos largos manda el **modo más lento**, λ_2 .”
- “El **criterio de Mpemba** es entonces puro álgebra: el peso inicial de cada modo es el solapamiento $a_k = \text{Tr}(l_k^\dagger \rho_0)$. Si conseguimos una preparación con **$a_2 = 0$** , esa preparación *se salta el modo lento* y relaja a la tasa rápida — por eso adelanta a otra que sí lo tiene.”
- “Un detalle técnico clave: $\mathcal{L}$ es **no normal**, sus modos *no* son ortogonales; eso es lo que permite que una curva que parte más lejos termine por debajo. Todo esto lo veremos reproducido numéricamente.”
- (*Transición*) “Lo primero, entonces: calcular esos autovalores. Tarea 1.”

Juan 2 · T1 Modos lentos — Método y discretización

1:10

- “Problema: el Liouvilliano es una matriz $4^N \times 4^N$. Diagonalizarla en denso es inviable ya para pocos espines.”
- “Solución: **Arnoldi**, pero con un truco. En vez de aplicar $\mathcal{L}$ (cuyos modos de interés son los *interiores*, cerca de 0, difíciles para Arnoldi), aplicamos el **propagador** $P = e^{\tau \mathcal{L}}$. Los modos lentos de $\mathcal{L}$ se vuelven los **dominantes** de P , que es justo lo que Arnoldi extrae primero — lo llamamos *faster than the clock*.”

- “Y la acción de P es **matrix-free**: integramos $\dot{V} = \mathcal{L}[V]$ con RK4, sin formar nunca la matriz gigante.”

Juan 2 · T1 — Núcleo serial | paralelo

0:50

- “El coste vive en el **producto de matrices** del SpMV. A la izquierda, serial: recorremos todas las filas.”
- “A la derecha, paralelo en **dos niveles**: cada **rango MPI** calcula un *bloque de filas* (paralelismo de datos), y dentro **OpenMP** reparte ese bloque entre hilos. Al final, **MPI_Allgatherv** reconstruye la matriz completa en todos los rangos.”

Juan 2 · T1 — Resultados

0:50

- “Validación: los autovalores lentos coinciden con la diagonalización densa con errores de 10^{-13} a 10^{-5} .”
- “Escalado $\sim 2.9\times$ con OpenMP y MPI. ¿Por qué no más? Porque el SpMV está **acoplado** y es *memory-bound*: los hilos compiten por el ancho de banda, y MPI paga la comunicación del **Allgatherv**. Es el caso *menos* paralelizable de los que veremos — y es un resultado, no un fallo.”
- “Y lo importante para la historia: T1 nos da el λ_2 y el a_2 que **determinan el cruce** de Mpemba.” (*pasa a Sebastián*)

Sebastián 3 · T2a Evolución RK4 — Método y discretización

1:10

- “Ahora la **evolución temporal**: dado un estado inicial, queremos la trayectoria $\rho(t)$ y su distancia al estacionario.”
- “Vía directa: **Runge–Kutta de orden 4** sobre $\dot{\rho} = \mathcal{L}[\rho]$. Cuatro evaluaciones de $\mathcal{L}$ por paso, también **matrix-free** (conmutadores y disipador como productos de matrices $d \times d$).”
- “¿Cómo paralelizar? Los pasos tienen **dependencia temporal estricta**: ρ_{n+1} necesita ρ_n . No se puede paralelizar *entre* pasos. El paralelismo está *dentro* del paso, en el producto de matrices. Datos regulares en un nodo $\Rightarrow$ el paradigma natural es **OpenMP**, no MPI (que pagaría comunicación en cada uno de los miles de pasos).”

Sebastián 3 · T2a — Núcleo serial | paralelo

0:50

- “El núcleo es el **matmul**. La versión paralela es *una sola línea* de diferencia: **#pragma omp parallel for**, que reparte las filas i entre hilos. Son independientes, no hay condiciones de carrera.”
- “Detalle bonito de ingeniería: es el *mismo* binario; cambiando la variable **OMP_NUM_THREADS** obtenemos las curvas serie vs paralelo, sin recompilar.”

Sebastián 3 · T2a — Resultados (computacional y físico)

0:55

- “Izquierda: tiempo serie vs paralelo según el tamaño. El speedup **crece con la malla** hasta $\sim 4\times$, pero es *sublineal*: otra vez *bandwidth-bound* más el coste de crear hilos en cada **matmul**.”
- “Derecha, lo físico: **aquí está el cruce**. La preparación que parte lejos (roja) **adelanta** a la cercana (azul) en $t^* \approx 0.69$. Validamos RK4 a orden 4 (pendiente ≈ 4.04).”

Sebastián 4 · T2b Krylov — Método y el resultado central

1:20

- “Segunda vía para la misma evolución, más sofisticada: en vez de dar muchos pasitos, aplicamos directamente la **acción del exponencial** $e^{\tau\mathcal{L}}$, que es la solución exacta en un paso τ .”
- “Pero $e^{\tau\mathcal{L}}$ es un objeto enorme. El truco de **Krylov**: proyectamos en un subespacio pequeño (Arnoldi, $m \sim 30$) y exponenciamos ahí una matriz $m \times m$ trivial.”

- “Comparte el *mismo* hotspot que RK4 (el `matmul`), así que la paralelización OpenMP y el escalado son equivalentes.”
- “El resultado central está en esta gráfica: **a igual trabajo, Krylov es unas $25\times$ más preciso** que RK4, y permite pasos de tiempo grandes siendo *incondicionalmente estable*. Es el método de elección para llegar al estacionario.”

Sebastián 4 · T2b — El cruce

0:35

- “Y reproduce el **mismo cruce**, $t^* \approx 0.69$, idéntico a RK4 — de hecho coinciden a 10^{-10} en la distancia final. Mensaje: el cruce es **física**, no un artefacto del método numérico.”

Sebastián 5 · T3 Trayectorias — Método y núcleo serial | paralelo

1:10

- “Hasta aquí trabajamos con ρ completo, de tamaño 4^N : inviable para **muchos cuerpos**. Cambio de paradigma: **trayectorias cuánticas** (Monte Carlo wavefunction).”
- “En vez de propagar ρ , propagamos M **estados puros** $|\psi\rangle$ de tamaño 2^N — la raíz — que evolucionan con un Hamiltoniano efectivo y dan **saltos** estocásticos. El promedio reconstruye ρ .”
- “La gran ventaja para HPC: cada trayectoria es **independiente** — *embarrassingly parallel*. MPI reparte las M entre rangos, cada uno calcula su lote *sin comunicación*, y **una sola MPI_Reduce** al final promedia.”

Sebastián 5 · T3 — Resultados

0:50

- “Y se nota: el escalado es **casi ideal**, $\sim 5.6\times$ y **constante** al crecer el trabajo — en claro contraste con el OpenMP sublineal de T2. Ese contraste es *el mensaje HPC de la charla: el patrón de cómputo dicta el paradigma*, no una preferencia.”
- “La precisión converge como $M^{-1/2}$. Y — otra vez — el **cruce de Mpemba**, ahora con el método estocástico, en $t^* \approx 0.75$ (la pequeña diferencia es el ruido Monte Carlo).”

Sebastián 5 · CUDA T4 — GPU

0:55

- “Como bonus, el **tercer escalón** de paralelización para la evolución densa: la GPU. El producto de matrices denso es el caso ideal para cuBLAS.”
- “En una **T4** de Colab: pierde para tamaños pequeños (overhead de lanzar kernels) pero **gana** $\sim 5\times$ en cuanto la malla crece — para $N = 9$, de 943 segundos en CPU a 190 en GPU. El producto puro llega a 250 GFLOP/s.”
- “Y validación triple: GPU = CPU = C++. El notebook ejecutado y los resultados están en el repositorio.” (*pasa a Juan*)

Juan 6 · T3b Redes tensoriales — Método y discretización

1:10

- “La otra gran ruta a muchos cuerpos, complementaria a las trayectorias: **redes tensoriales**.”
- “Escribimos la matriz densidad como un **superket** en forma de MPS — una cadena de tensores, con dimensión física 4 por sitio. La ecuación maestra se integra con **TEBD**: trotterizamos $e^{dt\mathcal{L}}$ en **puertas locales** de dos sitios y **truncamos** el enlace a χ con una SVD.”
- “Resultado: coste **polinómico en χ** , no exponencial en N . Eso abre tamaños imposibles en denso.”
- “Paralelización por **hilos**: en una capa de Trotter, los bonds pares (o impares) son *disjuntos*; sus SVD son independientes. Y numpy **libera el GIL** durante la SVD, así que los hilos escalan de verdad.”

- “Serial: un bucle que aplica las compuertas de los bonds una a una. Paralelo: un **executor.map** lanza las compuertas disjuntas en hilos.”
- “Es un paralelismo **estructurado** — ni tan acoplado como el SpMV de T1, ni tan libre como las trayectorias. Queda en medio, y eso se ve en el escalado.”

- “Validación: comparado con el resultado exacto a N pequeño, el error **cae al subir** χ , hasta 6×10^{-7} — control sistemático.”
- “Escala $\sim 3.6\times$ en hilos (entre T1 y las trayectorias, como anticipamos). Y el alcance: llegamos a $N = 32$ — un espacio de Hilbert de 10^{19} , absolutamente imposible en denso — evolucionado en segundos.”

- “Y cerramos con lo que une todo: **también las redes tensoriales reproducen el cruce**. Las mismas dos preparaciones, $|+\rangle$ que parte lejos adelanta a $|0\rangle$ en $t^* \approx 0.69$.”
- “Idéntico a RK4 y Krylov, porque a χ grande TEBD es exacto. Es decir: **los cuatro métodos de evolución** — RK4, Krylov, trayectorias y redes tensoriales — dan *el mismo* cruce. Validación cruzada total.”

- “**HPC**. El mensaje central: el *patrón de cómputo* determina la escalabilidad. Lo vimos como un espectro: *acoplado* (T1, SpMV, $\sim 3\times$) < *estructurado* (T3b, redes tensoriales, $\sim 3.6\times$) < *independiente* (trayectorias, $\sim 5.6\times$ casi ideal). Y la evolución densa, además, vuela en GPU.”
- “Cada paradigma — OpenMP, MPI, hilos, CUDA — lo elegimos por la *estructura del algoritmo*, no por moda.”
- “**Física**. El efecto Mpemba cuántico tiene origen **geométrico-espectral**: la no-normalidad de $\mathcal{L}$ más el criterio $a_2 = 0$. Lo **reproducimos con los cuatro métodos** ($t^* \approx 0.69\text{--}0.75$), validados de forma cruzada, y escalado a muchos cuerpos.” (*pasa a Sebastián*)

- “Para cerrar, por qué importa. La **firma es universal**: el mismo mecanismo de *geometría espectral de la relajación* explica el cruce desde el macro clásico (agua, coloides) hasta lo cuántico. Es el mismo lenguaje matemático.”
- “Lo que devuelve el enfoque cuántico al problema clásico: un **criterio operacional** — $a_2 = 0$ — y las **herramientas HPC** para *predecir* y *diseñar* el efecto en vez de buscarlo a ciegas.”
- “Con aplicaciones concretas: **termometría cuántica** y protocolos de **enfriamiento acelerado**.”
- “Y esa es la idea: una anomalía vieja de la física, entendida como álgebra de autovalores y llevada a HPC. Muchas gracias.” (*cierre*)

Notas.

- Énfasis recurrente: los **cuatro cruces** (T2a, T2b, T3, T3b dan el mismo t^*) y el contraste de escalado **acoplado** < **estructurado** < **independiente**.
- Si vas sobre tiempo: la slide CUDA es la más prescindible; las ecuaciones de la slide matemática se pueden resumir.
- Frase ancla: “**el patrón de cómputo dicta el paradigma**”.